

Algorithms for Data Analysis and Data Mining

Natalia Bardadyn, Karolina Dadej, Joanna Dagil, Marcin Przesmycki

June 1, 2026

A. Data Analysis and Regression

1. Logistic Regression

(a)

The analysis of the full dataset was performed using a logistic regression model:

$$\log\left(\frac{p}{1-p}\right) = \beta_0 + \beta_1 \cdot \text{hours}$$

The threshold for when the function reaches 0.5 and thus the binary output returns 1 was calculated using the following formula.

$$0.5 = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x)}}$$

$$1 + e^{-(\beta_0 + \beta_1 x)} = 2$$

$$e^{-(\beta_0 + \beta_1 x)} = 1$$

$$-(\beta_0 + \beta_1 x) = 0$$

$$x = -\beta_0 / \beta_1$$

The results show a clear relationship between study hours and the probability of success. The model provides a good fit to the data and achieves classification performance using the chosen decision threshold.

Final model function that resulted:

$$\log\left(\frac{p}{1-p}\right) = -10.83988 + 0.2091565 \cdot \text{hours}$$

Final threshold for when the function reaches 0.5 probability:

$$51.82663$$

(b)

The analysis of the dataset with last point removed was performed using the same model and metrics.

$$\log\left(\frac{p}{1-p}\right) = -10.83733 + 0.2091066 \cdot \text{hours}$$

Final threshold for when the function reaches 0.5 probability:

$$51.82679$$

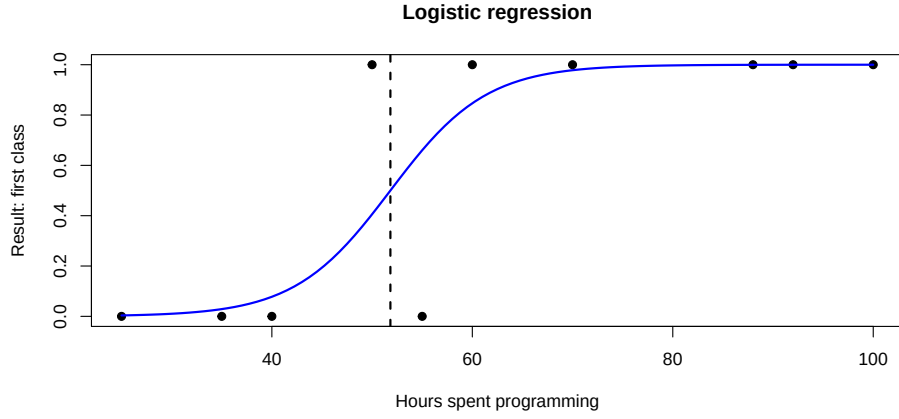


Figure 1: Logistic regression

Removing the point (100,1) does not significantly change the result. The fitted curve and the threshold remain almost the same. This is because the point (100,1) is consistent with the general pattern in the data: students with high numbers of hours usually obtained result 1.

The model already has several high-hour observations with result 1, for example 70, 88 and 92 hours. Therefore, the point (100,1) does not add much new information to the model.

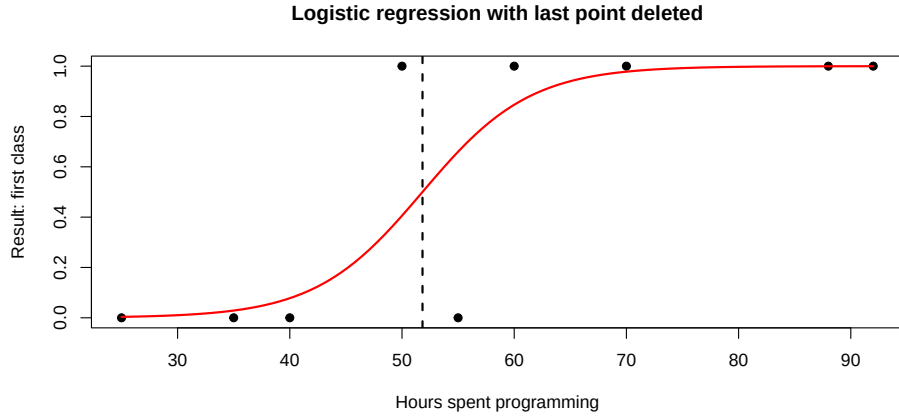


Figure 2: Logistic regression with last point deleted

(c)

The analysis of the dataset with added data point was performed using the same model and metrics.

$$\log\left(\frac{p}{1-p}\right) = -10.84002 + 0.2091594 \cdot \text{hours}$$

Final threshold for when the function reaches 0.5 probability

$$51.82662$$

Adding the point (115,1) very slightly affects the estimated model. The coefficient associated with the predictor becomes greater, indicating a steeper increase in the probability of success.

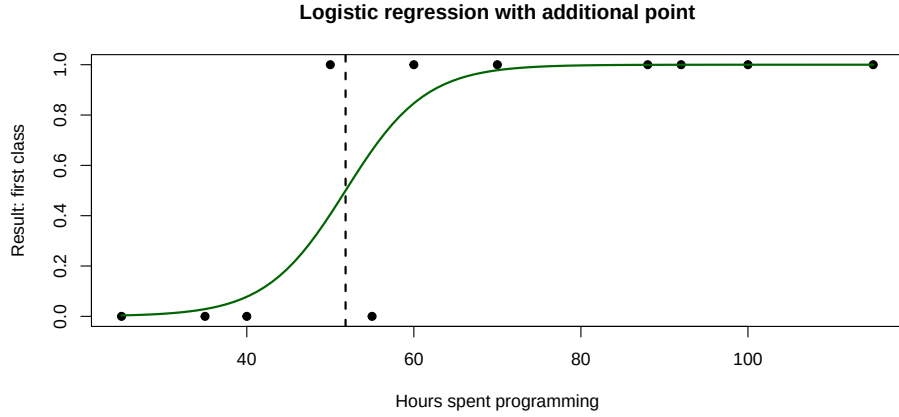


Figure 3: Logistic regression with additional point

Table 1: Comparison of logistic regression models

model	threshold	beta0	beta1
Original	51.8266	-10.8399	0.2092
Without (100,1)	51.8268	-10.8373	0.2091
With (115,1)	51.8266	-10.8400	0.2092

(d)

Logistic regression is a simple classification method for binary outcomes. One of its main advantages is that the estimated coefficients can be interpreted in terms of log-odds and odds ratios, which makes the relationship between predictors and the response variable easier to understand. Another important advantage is that the model produces predicted probabilities in the range:

$$0 \leq p \leq 1$$

making it a valuable method for probabilistic interpretation and decision-making problems. It is also computationally efficient and performs well on relatively small datasets. Lastly, it does not require the predictor variables to follow a normal distribution.

A major limitation of logistic regression is the assumption of a linear relationship between the predictors and the log-odds of the outcome:

$$\log \left(\frac{p}{1-p} \right) = \beta_0 + \beta_1 x$$

If the true relationship is non-linear, the model may provide poor predictive performance. This method can also be sensitive to outliers and influential observations, especially in small datasets although the addition of the point (115, 1) caused only a small change in the estimated coefficients and decision boundary. Finally, logistic regression may perform worse than more advanced machine learning methods when dealing with complex decision boundaries or large numbers of interacting variables.

2. Nonlinear Least-Squares

(a)

(b)

(c)

(d)

B. Given Dataset

3. Data analysis

We have decided to work on the dataset `sggw_c.csv` and to perform SVM analysis.

(a) Data summary and visualization

Descriptive statistics of the data include mean, median, variance, as well as minimum and maximum.

Several variables are categorical or ordinal variables encoded as numerical values. Therefore, descriptive statistics such as mean and variance, as well as histograms and boxplots below, should be interpreted with caution.

variable	min	max	mean	median	variance
Account.Status..with.the.bank.	1	4	2.49	2.00	1.51
Credit.Months	6	60	20.07	18.00	135.10
Credit.Amount..K.1000.	3	159	31.38	21.00	746.60
Saving..account..bonds.	1	5	2.14	1.00	2.44
Years.of.Employment	1	5	3.56	3.00	1.39
Residence.Years	1	4	2.81	3.00	1.19
Age	20	74	36.43	34.00	150.76
House..1.own..2.rent.	1	4	1.44	1.00	0.36
Existing.credit.installment	1	2	1.15	1.00	0.13
Job..1.Good.	1	2	1.48	1.00	0.25
Dependent..e.g...children.	1	2	1.05	1.00	0.04
Credit..1.Good.	1	2	1.28	1.00	0.20

Table 2: Descriptive statistics of the dataset.

The data were visualized using both histograms and boxplots.

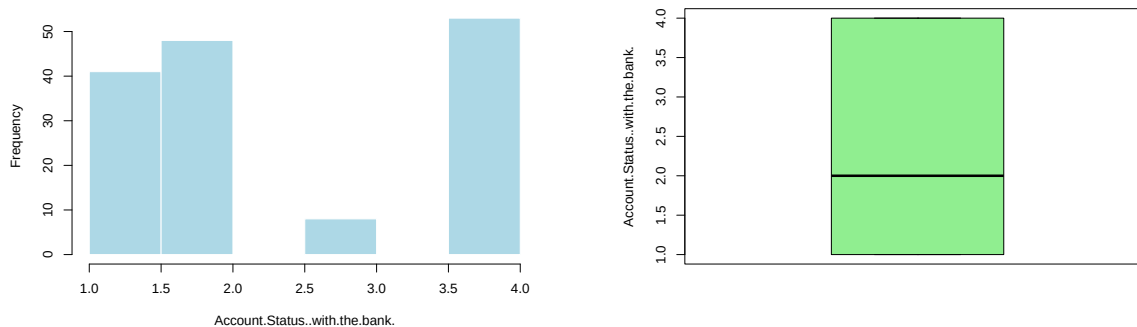


Figure 4: Distribution of Account Status with the bank

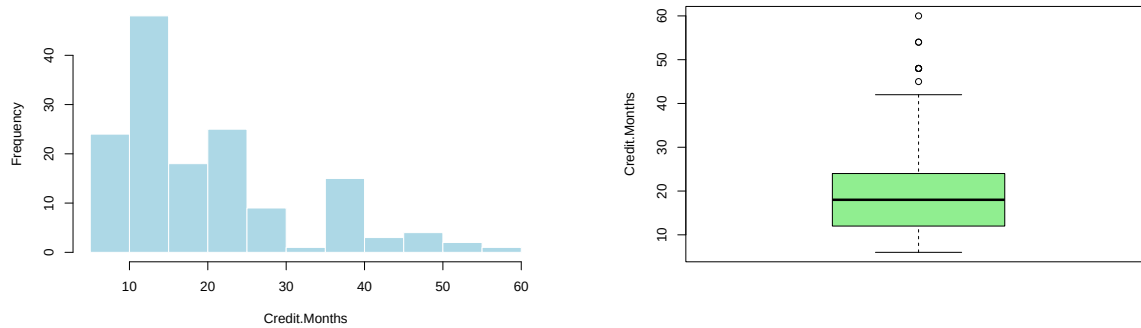


Figure 5: Distribution of Credit Months

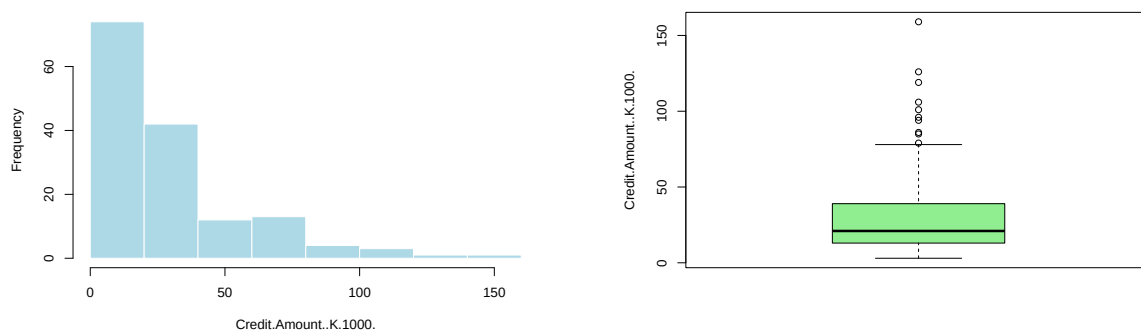


Figure 6: Distribution of Credit Amount

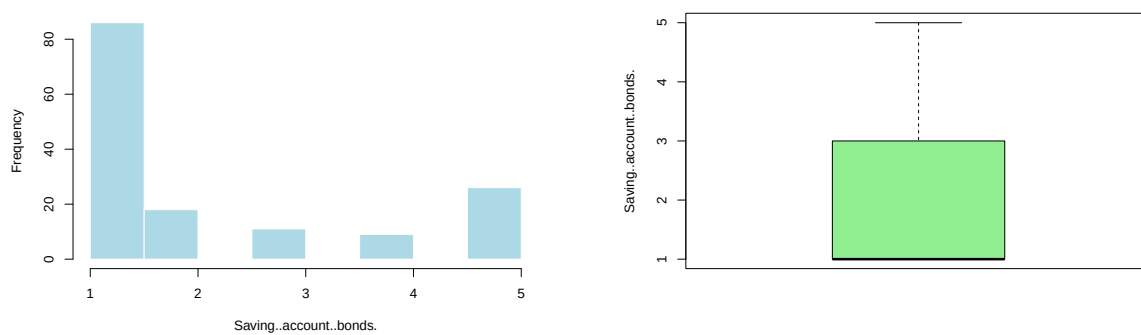


Figure 7: Distribution of Saving account bonds

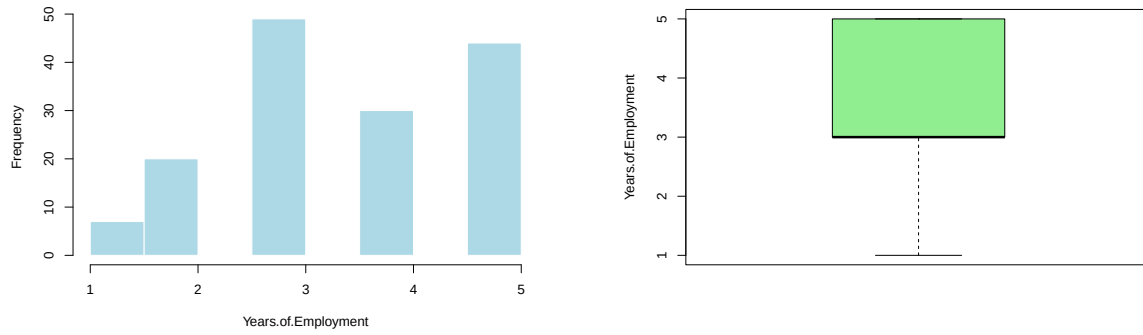


Figure 8: Distribution of Years of Employment

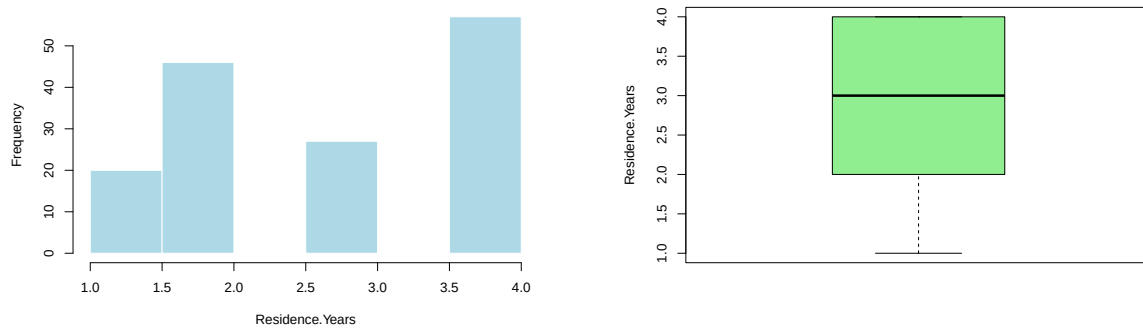


Figure 9: Distribution of Residence Years

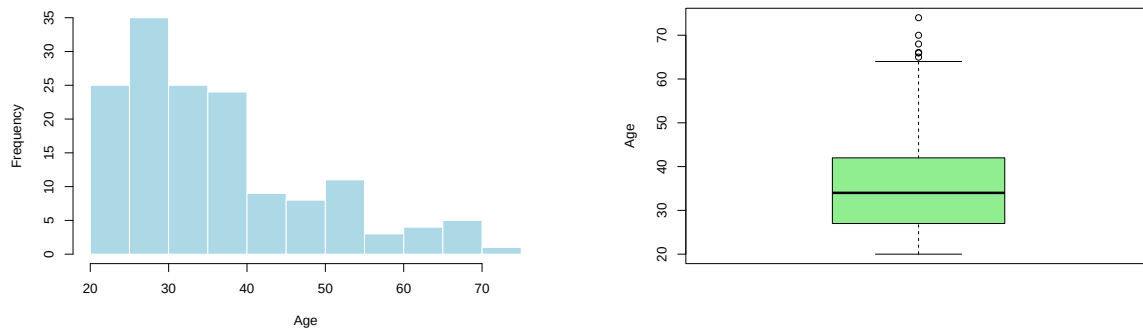


Figure 10: Distribution of Age

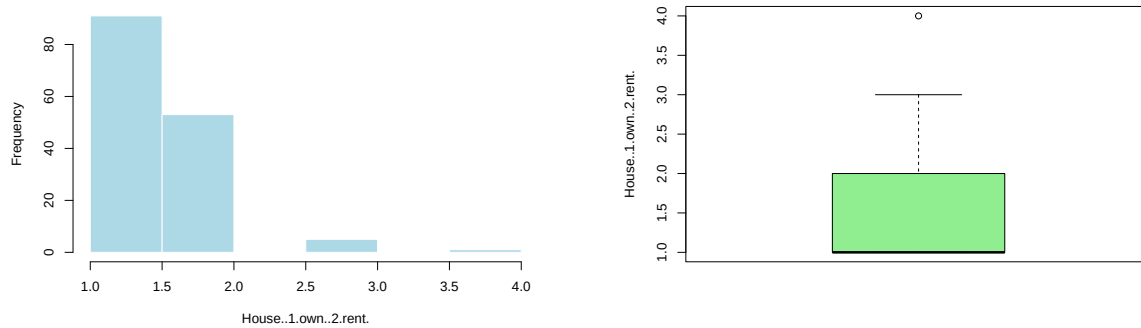


Figure 11: Distribution of House Ownership or Rental

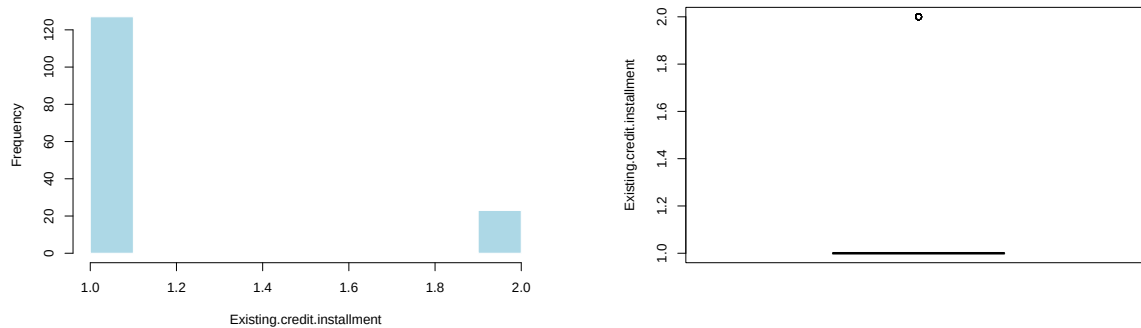


Figure 12: Distribution of Existing Credit Installments

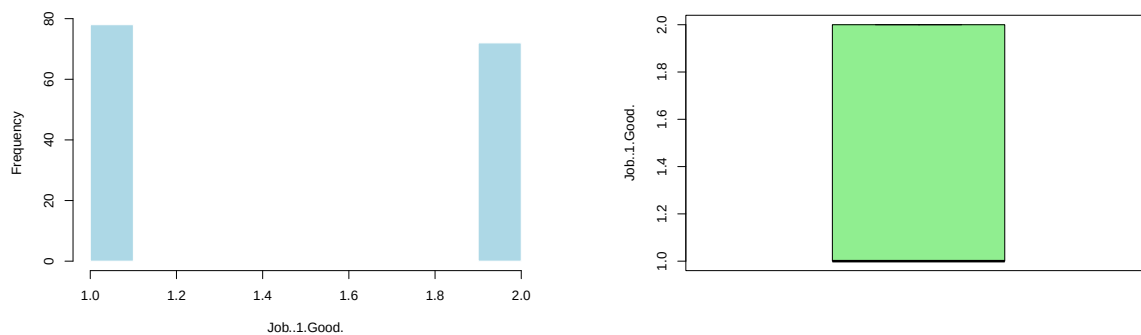


Figure 13: Distribution of Job

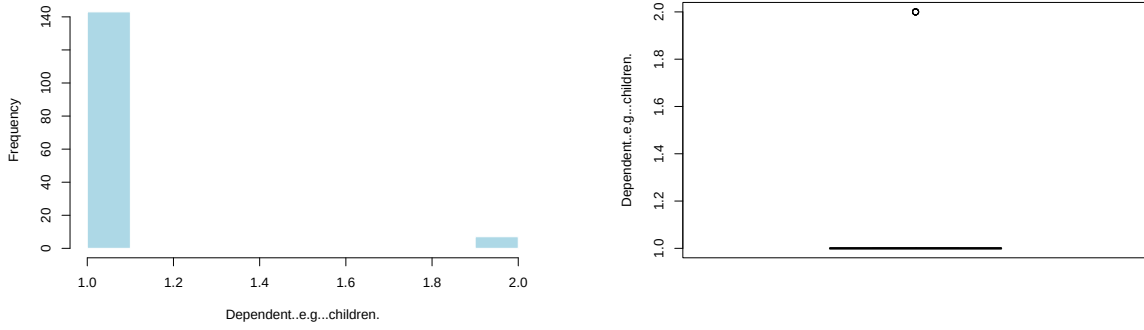


Figure 14: Distribution of Dependent

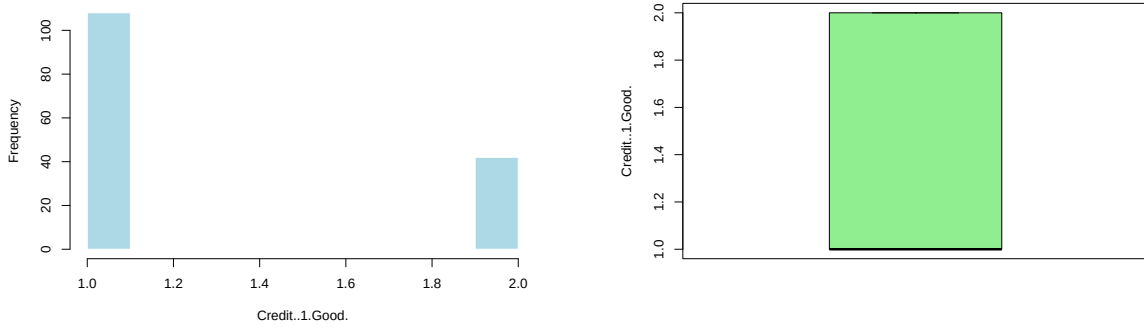


Figure 15: Distribution of Credit

(b) SVM analysis

We have chosen to perform our analysis using Support Vector Machine.

Code is heavily commented but there are a few points worth discussing further.

The dataset was split into training and test sets using an 80 : 20 proportion. Since the amount of input data was limited - 150 entries with 108 **Good** labels and 42 **Bad** labels, we decided to check the proportions of **Good** and **Bad** labels in the sampling split. With the random seed set to 42 the split was as below:

Dataset	class	count	proportion
Full dataset	Good	108	0.72
Full dataset	Bad	42	0.28
Training set	Good	87	0.72
Training set	Bad	33	0.28
Test set	Good	21	0.70
Test set	Bad	9	0.30

Table 3: Class distribution in the full dataset, training set and test set.

As it can be seen, the proportions of the original dataset are similar to the proportions in train and in test datasets, thus the split can be described as representative of the original dataset.

The standard SVM model was trained using a radial kernel. The target variable was converted to a factor before training, so the model performed classification rather than regression. The predictors were scaled, which is important for SVM because the method is sensitive to the relative scale of input variables.

(c) Results

The SVM model was trained as follows:

```
model <- svm(  
  x = data_train_x,  
  y = data_train_y,  
  kernel = "radial",  
  scale = TRUE)
```

This resulted in the following predictions:

Predicted	Good	Bad
Good	21	8
Bad	0	1

Table 4: Confusion matrix for the SVM model.

As we can see, the model works but it performs barely better than a naive model predicting only **Good** label for all cases, having accuracy at 0.7333 as compared to naive accuracy at 0.7. This is probably caused by the small dataset size, the use of default SVM parameters and class imbalance, having most of the data labeled as **Good**. Since the problem is credit risk, we could sensibly assume that the more optimal would be a model that makes less mistakes in labeling **Bad** cases as **Good** even if it has an overall lower accuracy. A **Bad** case predicted as **Good** is the dangerous error in credit risk.

We can address this by using class weights that increase the penalty for false **Good** predictions.

```
class_weights <- c(  
  Good = 1,  
  Bad = sum(data_train_y == "Good") / sum(data_train_y == "Bad")  
)  
  
model_weighted <- svm(  
  x = data_train_x,  
  y = data_train_y,  
  kernel = "radial",  
  scale = TRUE,  
  class.weights = class_weights  
)
```

The weighted model, evaluated on the same train/test split, performed as follows.

Predicted	Good	Bad
Good	16	6
Bad	5	3

Table 5: Confusion matrix for the SVM model with weighted classes.

The resulting overall accuracy is lower, but the weighted model reduces the number of **Bad** cases incorrectly classified as **Good**. Depending on the preferred types of errors the weights can be further adjusted. Below are the final metrics of the predictions of both SVM models.

	Accuracy	Precision	Bad	Recall	Bad	Specificity	F1	Bad
Standard_SVM	0.73		1.00		0.11	1.00		0.20
Weighted_SVM	0.63		0.38		0.33	0.76		0.35

Table 6: Comparison of evaluation metrics for standard and weighted SVM models.

Since the dataset is small and only one train/test split was used, the results should be interpreted as exploratory. A more reliable evaluation could use cross-validation.

C. Mini-Project